AGH University of Krakow

Mobile App Development

Laboratory 5 Report

Static Layouts in Android

Student:	Carlos Guzman
Project name:	CarlosGuzmanLayoutsAndActions
Language:	Kotlin
Development environment:	Android Studio
Date:	April 22, 2026

1. Objective

The objective of this laboratory practice is to create static user interfaces in Android Studio using XML layout files. During the practice, different types of layouts were explored, including `RelativeLayout` and `LinearLayout`, as well as the use of interface resources such as colors, strings, and drawable images.

2. General Description

In this laboratory, an Android application project was created in Kotlin. The practice focused on editing the XML files directly in order to understand how static layouts are defined in Android. Different interface elements such as text fields, images, and buttons were positioned inside the layouts. In addition, custom colors and string resources were declared in the corresponding files under the `res` directory.

At the current stage, the project includes the following layout files:

- `activity_main.xml`
- `layout_weekend.xml`
- `layout_calculator.xml`

3. Resource Files

3.1. `colors.xml`

The following file defines the colors used in the layouts.

```xml
<?xml version="1.0" encoding="utf-8"?>
<resources>
    <color name="black">#FF000000</color>
    <color name="white">#FFFFFFFF</color>
    <color name="layout_background">#2196F3</color>
</resources>
```

3.2. `strings.xml`

The following file defines the text resources used by the application.

```xml
<resources>
    <string name="app_name">CarlosGuzmanLayoutsAndActions</string>
    <string name="hello_string">Hello There&#10;From XML File</string>
    <string name="android_is_so_awesome">Android is so&#10;awesome</string>
    <string name="remeber">Remember!</string>
</resources>
```

4. Main Layout: `activity_main.xml`

The first part of the practice consists of creating a static screen using a `RelativeLayout`. A custom background color was added and two text elements were positioned on the screen. These elements were configured using XML attributes such as width, height, text size, text style, alignment, margins, and background colors.

This layout was used to understand how elements can be arranged relative to the parent container and to each other.

4.1. XML Code

```xml
<?xml version="1.0" encoding="utf-8"?>
<RelativeLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:id="@+id/main"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:background="@color/layout_background">

    <EditText
        android:id="@+id/hello_there_edittext"
        android:layout_width="140dp"
        android:layout_height="wrap_content"
        android:text="@string/hello_string"
        android:textSize="18sp"
        android:textStyle="bold"
        android:background="@android:color/holo_green_light"
        android:textColor="@android:color/black"
        android:gravity="center"
        android:padding="6dp"
        android:layout_centerHorizontal="true"
        android:layout_centerVertical="true"
```

```
        android:layout_marginEnd="4dp" />

    <EditText
        android:id="@+id/awesome_android_edittext"
        android:layout_width="140dp"
        android:layout_height="wrap_content"
        android:text="@string/android_is_so_awesome"
        android:textSize="18sp"
        android:textStyle="bold"
        android:background="@android:color/holo_orange_dark"
        android:textColor="@android:color/black"
        android:gravity="center"
        android:padding="6dp"
        android:layout_toEndOf="@id/hello_there_edittext"
        android:layout_alignTop="@id/hello_there_edittext" />

</RelativeLayout>
```

4.2. Screenshot

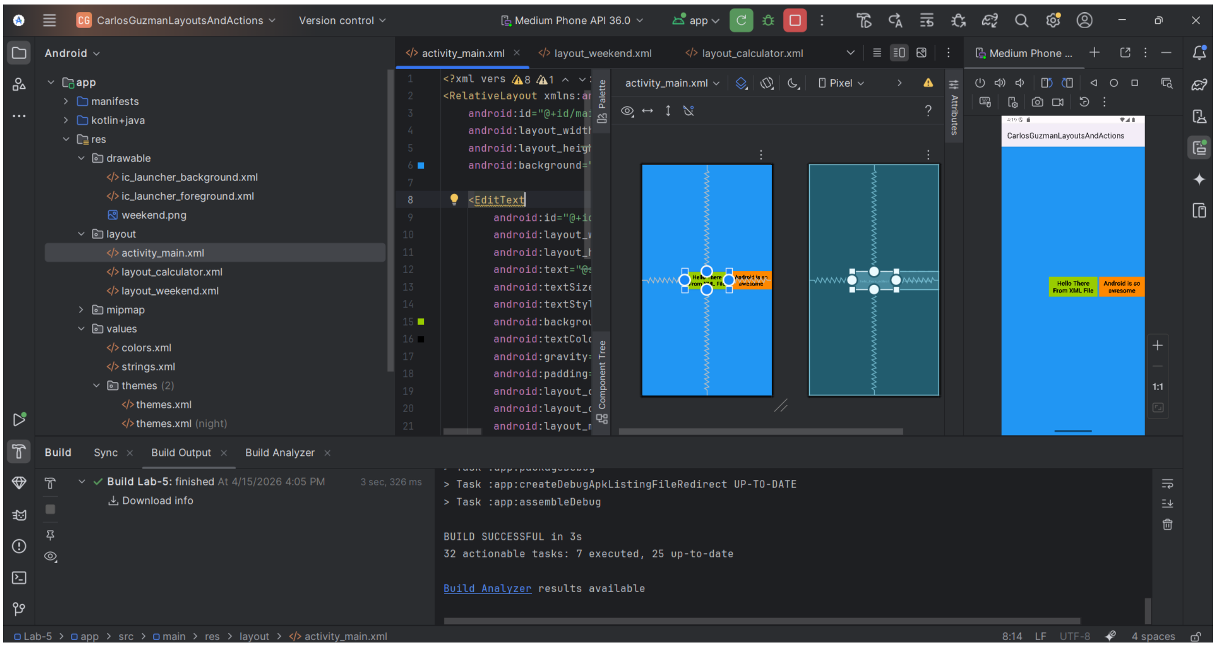

Figure 1: Main layout shown in the emulator.

5. Weekend Layout: `layout_weekend.xml`

The second part of the practice consists of creating a new XML layout file using a `LinearLayout`. In this screen, a title text and an image resource were added. This part of the laboratory helped to understand how to import drawable resources and how to organize interface elements vertically.

The image was added to the `drawable` folder and referenced inside the XML file. This layout is intended to show a simple reminder-style screen with a title and an image underneath.

5.1. XML Code

```xml
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:id="@+id/main"
    android:orientation="vertical"
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <EditText
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="@string/remeber"
        android:textSize="24sp"
        android:textStyle="bold"
        android:gravity="center"
        android:background="@android:color/holo_orange_dark"
        android:textColor="@color/white"
        android:padding="12dp" />

    <ImageView
        android:layout_width="match_parent"
        android:layout_height="0dp"
        android:layout_weight="1"
        android:src="@drawable/weekend"
        android:scaleType="centerCrop"
        android:contentDescription="Weekend image" />

</LinearLayout>
```

5.2. Screenshot

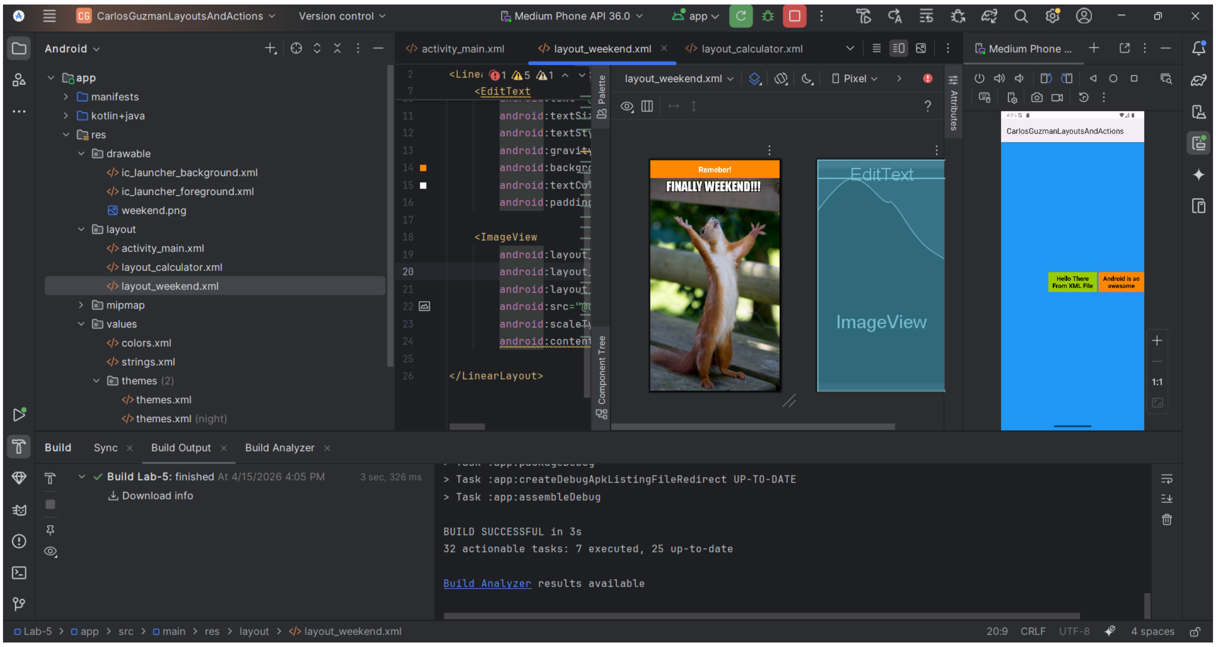

Figure 2: Weekend layout shown in the emulator.

6. Calculator Layout: `layout_calculator.xml`

The last part developed so far is a calculator-style interface made with XML. This layout was designed using nested `LinearLayout` containers in order to simulate rows of buttons and a display area at the top. It includes number buttons and operator buttons arranged in a grid-like structure.

This section demonstrates the use of multiple nested layouts, spacing, alignment, button styling, and a display area for the calculator.

6.1. XML Code

```xml
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:id="@+id/main"
    android:orientation="vertical"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:background="#FFFFFF"
    android:padding="16dp">

    <EditText
```

```
        android:id="@+id/display"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="5555"
        android:textSize="48sp"
        android:textStyle="bold"
        android:gravity="end"
        android:padding="16dp"
        android:inputType="none"
        android:background="@android:color/white" />

    <View
        android:layout_width="match_parent"
        android:layout_height="1dp"
        android:background="#CCCCCC"
        android:layout_marginBottom="16dp" />

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="horizontal"
        android:gravity="center"
        android:layout_marginBottom="8dp">

        <Button
            android:layout_width="70dp"
            android:layout_height="70dp"
            android:text="1"
            android:backgroundTint="#7B1FA2"
            android:textColor="@color/white"
            android:layout_margin="4dp"
            android:textSize="18sp" />

        <Button
            android:layout_width="70dp"
            android:layout_height="70dp"
            android:text="2"
            android:backgroundTint="#7B1FA2"
            android:textColor="@color/white"
            android:layout_margin="4dp"
            android:textSize="18sp" />
```

```
    <Button
        android:layout_width="70dp"
        android:layout_height="70dp"
        android:text="3"
        android:backgroundTint="#7B1FA2"
        android:textColor="@color/white"
        android:layout_margin="4dp"
        android:textSize="18sp" />

    <Button
        android:layout_width="70dp"
        android:layout_height="70dp"
        android:text="/"
        android:backgroundTint="#7B1FA2"
        android:textColor="@color/white"
        android:layout_margin="4dp"
        android:textSize="18sp" />
</LinearLayout>

<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="horizontal"
    android:gravity="center"
    android:layout_marginBottom="8dp">

    <Button
        android:layout_width="70dp"
        android:layout_height="70dp"
        android:text="4"
        android:backgroundTint="#7B1FA2"
        android:textColor="@color/white"
        android:layout_margin="4dp"
        android:textSize="18sp" />

    <Button
        android:layout_width="70dp"
        android:layout_height="70dp"
        android:text="5"
        android:backgroundTint="#7B1FA2"
```

```
            android:textColor="@color/white"
            android:layout_margin="4dp"
            android:textSize="18sp" />

        <Button
            android:layout_width="70dp"
            android:layout_height="70dp"
            android:text="6"
            android:backgroundTint="#7B1FA2"
            android:textColor="@color/white"
            android:layout_margin="4dp"
            android:textSize="18sp" />

        <Button
            android:layout_width="70dp"
            android:layout_height="70dp"
            android:text="-"
            android:backgroundTint="#7B1FA2"
            android:textColor="@color/white"
            android:layout_margin="4dp"
            android:textSize="18sp" />
    </LinearLayout>

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="horizontal"
        android:gravity="center"
        android:layout_marginBottom="8dp">

        <Button
            android:layout_width="70dp"
            android:layout_height="70dp"
            android:text="7"
            android:backgroundTint="#7B1FA2"
            android:textColor="@color/white"
            android:layout_margin="4dp"
            android:textSize="18sp" />

        <Button
            android:layout_width="70dp"
```

```
            android:layout_height="70dp"
            android:text="8"
            android:backgroundTint="#7B1FA2"
            android:textColor="@color/white"
            android:layout_margin="4dp"
            android:textSize="18sp" />

        <Button
            android:layout_width="70dp"
            android:layout_height="70dp"
            android:text="9"
            android:backgroundTint="#7B1FA2"
            android:textColor="@color/white"
            android:layout_margin="4dp"
            android:textSize="18sp" />

        <Button
            android:layout_width="70dp"
            android:layout_height="70dp"
            android:text="*"
            android:backgroundTint="#7B1FA2"
            android:textColor="@color/white"
            android:layout_margin="4dp"
            android:textSize="18sp" />
    </LinearLayout>

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="horizontal"
        android:gravity="center">

        <Button
            android:layout_width="70dp"
            android:layout_height="70dp"
            android:text="C"
            android:backgroundTint="#7B1FA2"
            android:textColor="@color/white"
            android:layout_margin="4dp"
            android:textSize="18sp" />

```

```
        <Button
            android:layout_width="70dp"
            android:layout_height="70dp"
            android:text="0"
            android:backgroundTint="#7B1FA2"
            android:textColor="@color/white"
            android:layout_margin="4dp"
            android:textSize="18sp" />

        <Button
            android:layout_width="70dp"
            android:layout_height="70dp"
            android:text="."
            android:backgroundTint="#7B1FA2"
            android:textColor="@color/white"
            android:layout_margin="4dp"
            android:textSize="18sp" />

        <Button
            android:layout_width="70dp"
            android:layout_height="70dp"
            android:text="="
            android:backgroundTint="#7B1FA2"
            android:textColor="@color/white"
            android:layout_margin="4dp"
            android:textSize="18sp" />
    </LinearLayout>

</LinearLayout>
```

6.2. Screenshot

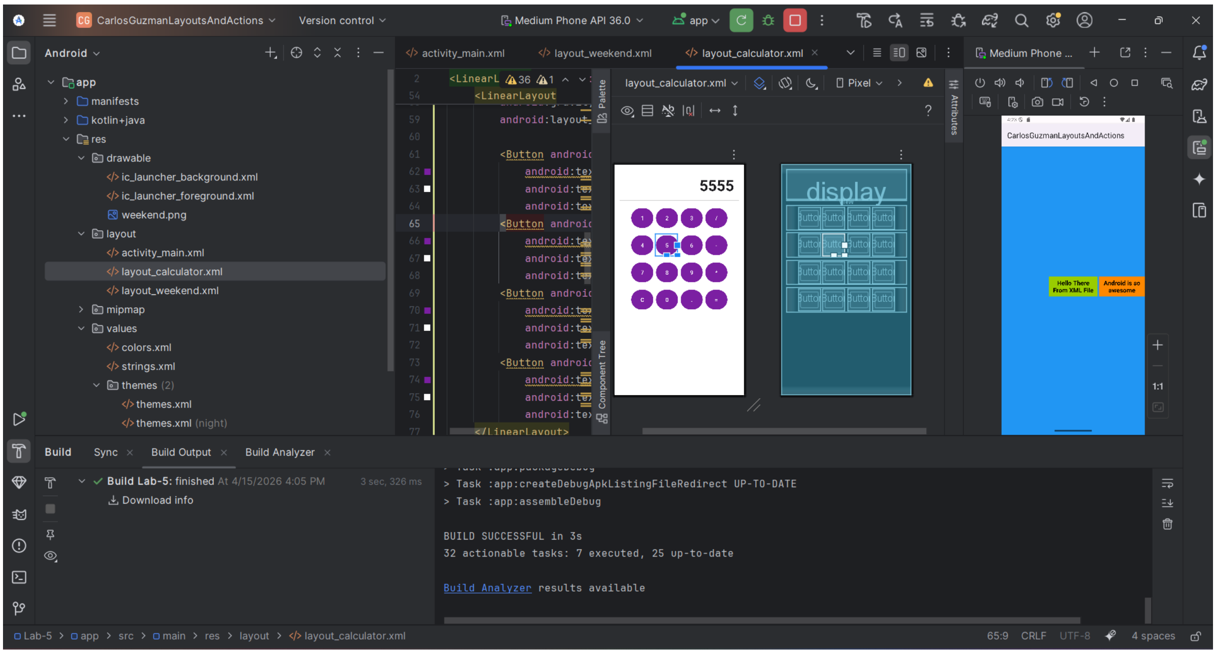

Figure 3: Calculator layout shown in the emulator.

7. MainActivity Configuration

To test each screen in the emulator, the `setContentView()` method in `MainActivity.kt` can be changed to load a specific XML layout.

7.1. `MainActivity.kt`

```
package com.example.carlosguzmanlayoutsandactions

import android.os.Bundle
import androidx.activity.enableEdgeToEdge
import androidx.appcompat.app.AppCompatActivity
import androidx.core.view.ViewCompat
import androidx.core.view.WindowInsetsCompat

class MainActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()

        // This line was changed depending on the layout being tested.
```

```
        // setContentView(R.layout.activity_main)
        // setContentView(R.layout.layout_weekend)
        setContentView(R.layout.layout_calculator)

        ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.main)) { v,
            insets ->
            val systemBars = insets.getInsets(WindowInsetsCompat.Type.systemBars
                ())
            v.setPadding(
                systemBars.left,
                systemBars.top,
                systemBars.right,
                systemBars.bottom
            )
            insets
        }
    }
}
```

The layout displayed in the emulator can be changed by replacing the active `setContentView()` line with one of the following options:

```
setContentView(R.layout.activity_main)
setContentView(R.layout.layout_weekend)
setContentView(R.layout.layout_calculator)
```

8. Theme Configuration

The following configuration was used to display the title bar of the application.

8.1. `themes.xml`

```
<resources xmlns:tools="http://schemas.android.com/tools">

    <style name="Base.Theme.CarlosGuzmanLayoutsAndActions"
        parent="Theme.Material3.DayNight">

        <item name="windowActionBar">true</item>
        <item name="windowNoTitle">false</item>

```

```
    </style>

    <style name="Theme.CarlosGuzmanLayoutsAndActions"
        parent="Base.Theme.CarlosGuzmanLayoutsAndActions" />

</resources>
```

9. Conclusion

In this laboratory practice, static Android layouts were created using XML files. Different types of containers were used, including `RelativeLayout` and `LinearLayout`. Resources such as colors, strings, and images were also integrated into the project.

Although the project is still being completed, the current progress already demonstrates the main concepts of XML-based interface design in Android Studio. The practice has helped reinforce the use of layout attributes, visual organization, and the relationship between resources and interface files.